

CS236

Lab Quiz 3 Grading Rubric

Q1

Testcases:

- p1a -- mmap + write into mmaped area + munmap (this must pass, to check for further testcases)

Expected output:

```
None
mmap successful
munmap successful
```

- p1b -- mmap + sbrk (not into mmaped area) + munmap + write into unmapped area

Expected output:

```
None
mmap successful
sbrk successful
munmap successful
Writing into unmapped area...
usertrap(): unexpected scause 0xf pid=3
          sepc=0x5a stval=0x7000
```

- p1c -- mmap + sbrk (into mmaped area) + munmap

Expected output:

```
None
mmap successful
sbrk unsuccessful
munmap successful
```

- p1d -- mmap + sbrklazy (into mmapped area) + munmap

Expected output:

```
None
mmap successful
sbrk unsuccessful
munmap successful
```

- p1e -- mmap + write + munmap + sbrk (sbrk into previously mmapped area)

Expected output:

```
None
mmap successful
munmap successful
sbrk successful
```

- p1f -- mmap + mmap + write + sbrk (into mapped area) + unmap + unmap + sbrk (into unmapped area)

Expected output:

```
None
mmap successful
mmap successful
sbrk unsuccessful
munmap successful
munmap successful
sbrk successful
```

- p1g -- mmap into heap area

Expected output:

```
None
mmap unsuccessful
```

```
munmap unsuccessful
```

- p1h -- mmap at a non page aligned address

Expected output:

None

```
mmap successful
```

- p1i -- munmap twice on mmaped memory

Expected output:

None

```
mmap successful
```

```
munmap successful
```

```
munmap unsuccessful
```

- p1i.c:

C/C++

```
#include "kernel/types.h"
```

```
#include "kernel/stat.h"
```

```
#include "user/user.h"
```

```
#define PGSIZE (1<<12)
```

```
int
```

```
main()
```

```
{
```

```
    int* topOfHeap = (int*)(sbrk(1*PGSIZE) + PGSIZE);
```

```
    int* mmap_addr = (int*)(topOfHeap + 2 * PGSIZE);
```

```
    if(mmap((uint64)mmap_addr) <= 0) {
```

```
        printf("mmap unsuccessful\n");
```

```

    } else {
        printf("mmap successful\n");
    }

    if(munmap((uint64) mmap_addr) <= 0) {
        printf("munmap unsuccessful\n");
    } else {
        printf("munmap successful\n");
    }

    if(munmap((uint64) mmap_addr) <= 0) {
        printf("munmap unsuccessful\n");
    } else {
        printf("munmap successful\n");
    }

    return 0;
}

```

Q2

Marks are given if the test cases pass and there is no hardcoding.

tc-cont (0.5 marks)

There should be a few "Hello from target", then a "Target started", then a few more "Hello from target", then "Child killed" and "Kill succeeded".

tc-stop (1 mark)

There should be a few "Hello from target", then a "Target stopped", then "Exiting" without any more "Hello from target".

tc-2cont (0.5 marks)

There should be a few "Hello from target", then a "Target started", then a few more "Hello from target", then "Sent start again", then a few more "Hello from target", then "Child killed" and "Kill succeeded".

tc-2stop (0.5 marks)

There should be a few "Hello from target", then a "Target stopped" and "Sent stop again", then "Exiting" without any more "Hello from target".

tc-stop-cont (1.5 marks)

There should be a few "Hello from target", then a "Target stopped", then "Target started", then a few more "Hello from target", then "Child killed" and "Kill succeeded". There should be no "Hello from target" between "Target stopped" and "Target started".

tc-kill (1.5 marks)

There should be a few "Hello from target", then a "Target stopped", then "Child Killed" and "Kill succeeded" without any more "Hello from target". Note that "Kill succeeded" should be there.

tc-multi-proc (1 mark)

There should be a few "Hello from target", then "Target stopped" and "Part 1 done". After that, there should be "Target started" and "Part 2 done". Then some more "Hello from target", then "Child Killed" and "Kill succeeded". There should not be any "Hello from target" between "Target stopped" and "Target started".

tc-inv-proc (1 mark)

Output should match the sample output exactly, except for a difference in PID. At least one test case other than tc-inv-proc and tc-inv-signal should have passed.

tc-inv-signal (1 mark)

Output should match the sample output exactly, except for a difference in PID. At least one test case other than tc-inv-proc and tc-inv-signal should have passed.

tc-pause (1.5 marks)

Output should match the sample output exactly, except for a difference in PID. Note: See the appendix for tc-pause and sample output.

Q3

Marks are given for the following testcases. 0 marks are given for a testcase if it doesn't compile, or deadlocks.

- 1) tc-slice - 2 marks
- 2) tc-vruntime - 1 marks
- 3) tc-fork - 1 marks
- 4) tc-wakeup - 1 marks
- 5) tc-policy - 5 marks

There are three groups of testcases. [1], [2, 3, 4], [5]. The next group of testcases is only checked if all testcases in the previous group pass. Only tc-policy is run with 5 cores, whereas all other testcases are run on a single core.

Appendix

tc-pause.c

```
C/C++
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/param.h"

int main()
{
    int pid = fork();
    if (pid < 0) {
        printf("[pid %d] Fork failed\n", getpid());
        exit(pid);
    }

    if (pid == 0) {
        printf("[pid %d] Sleeping\n", getpid());
        pause(30);
        printf("[pid %d] Waking up\n", getpid());
    } else {
```

```

    pause(10);

    if (signal(pid, SIGSTOP) == 0)
        printf("[pid %d] Target stopped\n", getpid());
    else {
        printf("[pid %d] Signal failed\n", getpid());
        exit(-1);
    }

    pause(50);

    if (signal(pid, SIGCONT) == 0)
        printf("[pid %d] Target started\n", getpid());
    else {
        printf("[pid %d] Signal failed\n", getpid());
        exit(-1);
    }

    int ret = wait(0);
    if (ret == pid)
        printf("[pid %d] Wait succeeded\n", getpid());
    else {
        printf("[pid %d] Wait failed\n", getpid());
        exit(ret);
    }
}
exit(0);
}

```

Sample output

None

```

[pid 4] Sleeping
[pid 3] Target stopped
[pid 3] Target started
[pid 4] Waking up
[pid 3] Wait succeeded

```